

EvoCracker

AI RESEARCH PROJECT

Phase 4 of 5 — AI Evolution Proof & Algorithm Comparative Analysis

PHASE 4

AI Evolution Proof

Algorithm comparative analysis & dynamic evidence of real-time adaptation

Deliverable Summary

- Replaced static accuracy framing with measurable live evolution evidence.
- Connected enemy runtime telemetry directly to dashboard proof views.
- Added comparative algorithm runtime metrics for transparent benchmarking.

Verification Checklist

- ✓ Evolution evidence uses live runtime values, not synthetic filler data.
- ✓ Comparative pathfinding metrics are measurable in active sessions.
- ✓ Demonstration flow is reproducible for evaluator review.

1. Replacing ML Accuracy Scores with Evolution Graphs

In a standard ML environment, accuracy scores (F1, precision) prove intelligence. In EvoCracker, intelligence is proven through Emergent Fitness and Genetic Shifts, visualized live in the AI Analytics Panel (accessible via the ``` key).

Evolution Dashboard

- **Generation Fitness Charts** — A bar chart tracking average, median, and max fitness across dungeon floors. A steady upward trend proves the Genetic Algorithm is successfully weeding out weak enemies.
- **Gene Averages** — Visualizes specific genetic traits (Speed, Vision, Aggression, Caution, Pack Tendency, Ambush, Patrol) mutating over time.
- **Iteration Proof** — Stats are extracted 100% authentically from living enemies. The Strength Index Delta shows exactly how much stronger the current generation's genome is versus the baseline.

Example

If a player rushes, the population's Speed and Aggression graphs rise over generations as slow enemies are killed off. If a player hides often, Vision and Persistence graphs spike as enemies adapt to search longer.

2. Authentic Telemetry Extraction

Metrics are pulled directly from living AI agents via `getAnalyticsSnapshot()` in `EnemyBase.ts` twice a second. To ensure graphs are not filler data, the system pulls:

- Exact physical states — current health, speed.
- Genetic weights — 0.0 to 1.0 trait values.
- Living performance data — damage dealt, unique tiles explored, time stuck.

3. Search Algorithm Comparative Analysis

Because enemies genetically evolve their preferred pathfinding algorithms, their performance must be benchmarked. The Pathfinding Tab in the Analytics Dashboard provides a live comparative analysis.

Metric	What It Measures
Average Compute Time (ms)	Real-time JavaScript execution cost of calculating a route to the player.
Average Nodes Expanded	Memory and search-space efficiency. A* predictably expands fewer nodes than BFS in open spaces.

By tracking `nodesExpanded` and `pathComputeTimeMs`, we can physically prove that as enemies evolve higher intelligence (e.g., mutating to prefer A* or Greedy BFS over DLS), their pathfinding becomes demonstrably faster and more memory-efficient.

4. How to Demonstrate

Recommended Evaluator Flow:

- 1 Start the game and complete a floor.
- 2 Open the AI panel with the backtick (`) key.
- 3 Open the Player tab to show the AI has classified the player's playstyle based on raw keystrokes, movement samples, and dominant zones.
- 4 Open the Evolution tab to display the fitness chart climbing over multiple generations, proving the enemies are learning.
- 5 Open the Pathfinding tab to show the real-time runtime comparison (Compute Time vs. Nodes Expanded) between active search algorithms on the grid.